

KỊCH BẢN THUYẾT TRÌNH

Đề tài: Xây dựng Hệ thống Chat Server đa người dùng theo kiến trúc Microservices

Môn học: SE330 – Ngôn ngữ lập trình JAVA

Vai trò	Họ và tên	MSSV	Phần trình bày
Người 1 (Mở đầu)	Tô Thành Nguyên	24521207	Slide 1, 2, 3
Người 2	Lê Vũ Hoàng Nguyên	24521182	Slide 4, 5, 10
Người 3	Nguyễn Quốc Nguyên	24521197	Slide 6, 7
Người 4 (Kết)	Nguyễn Thành Phát	24521310	Slide 8, 9, 11

Tổng thời lượng gợi ý: ~12–15 phút. Ký hiệu » là ghi chú sân khấu (thao tác/chuyển slide), phần chữ thường là lời thoại.

NỘI DUNG KỊCH BẢN

• Slide 1 — Trang bìa & Giới thiệu nhóm

Người trình bày: Tô Thành Nguyên | Thời lượng gợi ý: ~1 phút

» Đứng giữa, mở slide bìa, cả nhóm chào khán giả

Em xin kính chào thầy/cô và tất cả các bạn. Nhóm em gồm 4 thành viên: Tô Thành Nguyên, Lê Vũ Hoàng Nguyên, Nguyễn Quốc Nguyên và Nguyễn Thành Phát.

Hôm nay nhóm em xin trình bày đề án môn Ngôn ngữ lập trình Java, với đề tài “Xây dựng Hệ thống Chat Server đa người dùng theo kiến trúc Microservices”. Toàn bộ mã nguồn dự án nhóm em đã công khai trên GitHub như đường link trên màn hình.

Sau đây, em xin đi vào phần tổng quan của dự án.

• Slide 2 — Tổng quan về dự án

Người trình bày: Tô Thành Nguyên | Thời lượng gợi ý: ~1.5 phút

Về tổng quan, sản phẩm của nhóm là một nền tảng nhắn tin thời gian thực — tương tự Discord — cho phép người dùng tạo máy chủ cộng đồng, trò chuyện theo kênh và nhắn tin riêng. Dự án xoay quanh bốn điểm cốt lõi.

Thứ nhất, về kiến trúc: hệ thống được chia thành 12 microservices độc lập, chạy trên nền Java 17 và Spring Boot 3 — mỗi service đảm nhận một nghiệp vụ riêng.

Thứ hai, về tái sử dụng mã: nhóm áp dụng tư duy hướng đối tượng và tách một module dùng chung tên common-lib để chia sẻ các DTO, Enum và tiện ích giữa các service.

Thứ ba, về xử lý thời gian thực: nhóm kết hợp WebSocket và RabbitMQ để quản lý luồng tin nhắn đa kênh với độ trễ thấp.

Và thứ tư, về dữ liệu: nhóm áp dụng mô hình Database-per-Service cùng Spring Data JPA — mỗi service sở hữu cơ sở dữ liệu riêng, không truy vấn JOIN chéo lẫn nhau.

● Slide 3 — Tổng quan yêu cầu (Use Case)

Người trình bày: Tô Thành Nguyên | Thời lượng gợi ý: ~1.5 phút

» Chỉ vào sơ đồ phân cấp vai trò bên trái

Đây là sơ đồ use case tổng quan. Hệ thống có bốn cấp vai trò kế thừa quyền lẫn nhau: thấp nhất là Regular User — người dùng thông thường, rồi đến Server Member, Server Admin và cao nhất là Server Owner — chủ máy chủ.

Người dùng thông thường có thể quản lý hồ sơ, kết bạn, nhắn tin riêng và gia nhập máy chủ bằng mã mời. Thành viên trong máy chủ thì giao tiếp theo kênh thời gian thực và gửi tệp.

Ở cấp quản trị, Server Admin quản lý cấu trúc kênh và điều phối thành viên — gán vai trò, kick, ban; còn Server Owner có toàn quyền cấu hình cấp cao. Ngoài ra hệ thống còn có tiến trình nền tự động giám sát và ghi log kiểm toán.

Tiếp theo, để hiện thực các yêu cầu này, mời bạn Hoàng Nguyên trình bày nền tảng công nghệ nhóm sử dụng.

● Slide 4 — Nền tảng Công nghệ

Người trình bày: Lê Vũ Hoàng Nguyên | Thời lượng gợi ý: ~1.5 phút

Cảm ơn bạn. Về nền tảng công nghệ, hệ thống được tổ chức thành nhiều tầng rõ ràng.

Tầng giao diện (Client) là ứng dụng Desktop viết bằng Java Swing, giao tiếp với máy chủ qua HTTP và WebSocket. Mọi yêu cầu đi qua tầng biên là Spring Cloud Gateway, kết hợp Spring Security và xác thực JWT.

Trái tim của hệ thống là hệ sinh thái Spring Boot: nhóm dùng Java 17, Spring Cloud OpenFeign cho giao tiếp giữa các service, Spring Data JPA để truy cập dữ liệu và REST API làm giao thức chính.

Phân hạ tầng gồm RabbitMQ làm message broker cho luồng bất đồng bộ, MySQL làm cơ sở dữ liệu quan hệ. Về DevOps, nhóm dùng Docker Compose, Maven, Git, Jenkins cho CI/CD và Prometheus, Grafana để giám sát.

• Slide 5 — Sơ đồ kiến trúc tổng thể

Người trình bày: Lê Vũ Hoàng Nguyên | Thời lượng gợi ý: ~1.5 phút

» Chỉ tổng quát từ trái sang phải trên sơ đồ kiến trúc

Slide này thể hiện bức tranh toàn cảnh cách các thành phần kết nối với nhau. Bắt đầu từ bên trái: ứng dụng client gửi request vào Spring Cloud Gateway. Tại đây, bộ lọc JWT sẽ kiểm tra token trước khi cho phép đi vào bên trong.

Ở giữa là cụm 12 microservices: auth, user-profile, friend, server, channel, role, messaging, presence, file, notification và log service. Các service gọi nhau qua Feign theo kiểu đồng bộ, đồng thời đẩy sự kiện qua RabbitMQ theo kiểu bất đồng bộ.

Bên phải, mỗi service kết nối tới cơ sở dữ liệu MySQL riêng của mình — đúng theo mô hình Database-per-Service mà nhóm đã nói. Điều này giúp các service tách biệt hoàn toàn và có thể mở rộng độc lập.

• Slide 10 — Thư viện dùng chung (common-lib)

Người trình bày: Lê Vũ Hoàng Nguyên | Thời lượng gợi ý: ~1.5 phút

Một điểm nhấn về kỹ thuật Java là module common-lib — nơi nhóm áp dụng nguyên lý tái sử dụng mã. Cả 12 service đều import module Maven này, gồm ba nhóm thành phần.

Thứ nhất là các Data Transfer Object — ví dụ MessageDTO, AuthRequest — định nghĩa cấu trúc dữ liệu truyền tải thống nhất. Nhóm tận dụng tính năng Record của Java 17 để tạo các lớp immutable ngắn gọn.

Thứ hai là Enum và kiểu dữ liệu, ví dụ MessageType với các giá trị CHAT, JOIN, LEAVE — định nghĩa một lần duy nhất, đảm bảo type-safety và tránh lỗi gõ sai chuỗi String.

Thứ ba là các tiện ích bảo mật như SecurityUtil hay JsonMapper — các hàm static dùng chung, giúp tránh việc copy-paste code giữa các service.

Tiếp theo, mời bạn Quốc Nguyên đi sâu vào phần bảo mật và xác thực của hệ thống.

• Slide 6 — Bảo mật & Xác thực tập trung

Người trình bày: Nguyễn Quốc Nguyên | Thời lượng gợi ý: ~1.5 phút

Cảm ơn bạn. Bảo mật là phần nhóm đặc biệt chú trọng, dựa trên Spring Security, BCrypt và JSON Web Token. Em xin trình bày bốn lớp bảo vệ.

Đầu tiên là mã hóa mật khẩu: nhóm dùng BCryptPasswordEncoder, nên mật khẩu không bao giờ được lưu dạng plain-text trong MySQL — ngăn rủi ro rò rỉ dữ liệu nếu cơ sở dữ liệu bị lộ.

Thứ hai là xác thực không trạng thái — Stateless JWT. Token chứa UserID và có thời hạn 2 giờ. Vì server không lưu session, hệ thống dễ dàng mở rộng theo chiều ngang.

Thứ ba và thứ tư là cơ chế bảo vệ tại Gateway: nhóm tạo một GlobalFilter — gọi là JwtAuthFilter — chặn mọi request ở tầng biên để giải mã và kiểm tra JWT. Chỉ khi token hợp lệ, request mới được forward xuống các microservice nội bộ, đảm bảo an ninh tuyệt đối cho toàn hệ thống.

• Slide 7 — Giao tiếp đồng bộ (OpenFeign)

Người trình bày: Nguyễn Quốc Nguyên | Thời lượng gợi ý: ~1.5 phút

Khi đã vào được bên trong, các service cần gọi lẫn nhau. Nhóm dùng Spring Cloud OpenFeign — một Declarative REST Client — giúp gọi API của service khác như gọi một hàm nội bộ.

Về bản chất, nhóm chỉ cần khai báo một Interface trong Java và đánh dấu bằng annotation @FeignClient. Khi chạy, Spring Cloud sẽ tự động sinh ra mã proxy để thực hiện HTTP request — lập trình viên không phải viết code gọi HTTP thủ công.

» Lấy ví dụ thực tế trong dự án

Ví dụ trong dự án: messaging-service gọi server-service để kiểm tra quyền thành viên trước khi cho phép gửi tin nhắn. Hay khi tạo một server mới, server-service gọi channel-service để tự động tạo kênh #general mặc định.

Đó là giao tiếp đồng bộ. Còn với những tác vụ cần độ trễ thấp và xử lý bất đồng bộ, mời bạn Thành Phát trình bày.

• Slide 8 — Xử lý thời gian thực (WebSocket)

Người trình bày: Nguyễn Thành Phát | Thời lượng gợi ý: ~1.5 phút

Cảm ơn bạn. Để có trải nghiệm chat tức thời, nhóm dùng WebSocket — một kết nối liên tục, hai chiều giữa client và server, thay vì liên tục hỏi-đáp như HTTP thường.

Về quản lý phiên kết nối, nhóm dùng cấu trúc ConcurrentHashMap trong Java để lưu danh sách các WebSocketSession đang online, ánh xạ theo ID người dùng. Việc dùng ConcurrentHashMap đảm bảo an toàn khi nhiều luồng truy cập cùng lúc.

Về luồng xử lý: client gửi một chuỗi JSON chứa metadata như channelId và type — ví dụ CHAT hoặc TYPING. Server Java parse JSON bằng Jackson, lưu vào cơ sở dữ liệu, rồi broadcast — phân phát lại — thông điệp cho tất cả các session thuộc cùng một kênh.

Toàn bộ logic này nằm trong class ChatWebSocketHandler, kế thừa từ TextWebSocketHandler của Spring, chịu trách nhiệm quản lý vòng đời của kết nối.

• Slide 9 — Giao tiếp bất đồng bộ (RabbitMQ)

Người trình bày: Nguyễn Thành Phát | Thời lượng gợi ý: ~1.5 phút

Bên cạnh giao tiếp tức thời, nhóm áp dụng kiến trúc hướng sự kiện — Event-Driven — với RabbitMQ. Mục tiêu là giảm tải cho service chính, tách các tác vụ phụ trợ ra khỏi luồng nghiệp vụ chính.

Nhóm dùng mô hình Topic Exchange, định tuyến tin nhắn linh hoạt dựa trên Routing Key.

Ví dụ với routing key log.# : messaging-service đẩy sự kiện chat, còn log-service lắng nghe và ghi ra file để kiểm toán — mà không làm chậm việc gửi tin của người dùng.

Hay với routing key notify.# : khi có tin nhắn mới, sự kiện được đẩy đến notification-service để tính số tin chưa đọc hoặc xử lý @mention. Nhờ vậy, người gửi nhận phản hồi ngay lập tức, còn các xử lý phụ diễn ra ngầm phía sau.

• Slide 11 — Kết thúc & Cảm ơn

Người trình bày: Nguyễn Thành Phát (đại diện nhóm) | Thời lượng gợi ý: ~1 phút

» Cả nhóm cùng đứng lên

Như vậy, nhóm em đã trình bày toàn bộ hệ thống Chat Server Microservices: từ tổng quan, kiến trúc, bảo mật, đến các cơ chế giao tiếp đồng bộ và bất đồng bộ trong Java.

Qua đề tài này, nhóm đã vận dụng được nhiều kiến thức cốt lõi của Java như lập trình hướng đối tượng, đa luồng, lập trình mạng và thiết kế hệ phân tán.

Phần trình bày của nhóm em đến đây là kết thúc. Cảm ơn thầy/cô và các bạn đã lắng nghe. Nhóm em rất mong nhận được câu hỏi và góp ý ạ.

Gợi ý khi thuyết trình

- Mỗi người luyện trước phần của mình để bám sát thời lượng; tổng buổi khoảng 12–15 phút.
- Khi chuyển người, đọc đúng câu “bàn giao” đã in cuối mỗi phần để liền mạch.
- Với các slide sơ đồ (3, 4, 5), vừa nói vừa dùng con trỏ chỉ vào thành phần đang nhắc đến.
- Chuẩn bị trước 2–3 câu hỏi thường gặp: vì sao chọn microservices, JWT hết hạn xử lý ra sao, RabbitMQ khác WebSocket thế nào.

PHỤ LỤC KỸ THUẬT — PHÒNG THỦ Q&A

Phần này tổng hợp mọi kỹ thuật thực sự dùng trong mã nguồn (kèm tên class/annotation/cấu hình thật) để trả lời khi hội đồng hỏi sâu. Không cần đọc khi thuyết trình — chỉ tra cứu khi cần.

A. Kiến trúc tổng thể & Build

- **Java 17 (LTS), Spring Boot 3.2.4, Spring Cloud 2023.0.1, Lombok 1.18.30.**
- Dự án **Maven đa mô-đun**: 1 POM cha + 13 mô-đun con + thư viện dùng chung *common-lib*; mỗi service là 1 artifact JAR độc lập.
- **12 microservices** + hạ tầng (MySQL 8.0, RabbitMQ 3, MinIO). Mô hình **Database-per-Service** — mỗi service một schema MySQL riêng, không JOIN chéo.
- Công tiêu biểu: gateway 8080, auth 8081, messaging 8082, presence 8083, log 8084, server 8085, channel 8086, notification 8088, file 8089, profile 8090, role 8091, friend 8092.

B. Bảo mật & Xác thực

- **Mã hóa mật khẩu**: *BCryptPasswordEncoder* của Spring Security (strength mặc định 10, tự sinh salt). Mật khẩu lưu dạng hash, không bao giờ plain-text.
- **JWT**: thư viện *JJWT 0.12.5*, ký bằng *HMAC-SHA* (Keys.hmacShaKeyFor). Claim chính là *subject = username*. Access token hết hạn **2 giờ**, refresh token **7 ngày**.
- **Refresh token**: lưu trong bảng *refresh_tokens* (token, userId, expiryDate); khi access token hết hạn, client dùng refresh token để lấy token mới.
- **Gateway lọc JWT**: class *JwtAuthFilter* là một *GlobalFilter* (order -1, chạy đầu tiên). Bỏ qua */api/auth/***; với request khác: đọc token từ header Bearer hoặc query param, kiểm tra chữ ký + hạn, rồi **inject header X-User-Id / X-Username** cho service nội bộ; sai thì trả 401.
- **Stateless**: server không lưu session → dễ scale ngang. CSRF disabled vì là REST API.
- **RBAC bitmask** (role-service): mỗi *Role* có trường *permissionBitmask* (int), mỗi bit = 1 quyền. Kiểm tra quyền bằng phép AND bit: $(bitmask \& PERMISSION_BIT) \neq 0$. Role gắn theo *serverId* (không global), có priority & bảng ban (BannedMember).

C. Giao tiếp đồng bộ — OpenFeign

- **Spring Cloud OpenFeign**: khai báo Interface + *@FeignClient(name, url)*, Spring sinh proxy HTTP lúc runtime. Bật bằng *@EnableFeignClients*.
- Các client thực tế: server→channel (tạo/lấy kênh), server→role & channel→role (kiểm tra quyền), messaging→server (kiểm tra thành viên trước khi gửi), messaging→role (quyền sửa/xóa tin).

D. Realtime — WebSocket

- Class *ChatWebSocketHandler* kế thừa *TextWebSocketHandler*; bật bằng *@EnableWebSocket*, đăng ký endpoint */ws/chat*.
- **Quản lý phiên**: *Map<String, WebSocketSession> = new ConcurrentHashMap<>()* (key = username) — thread-safe khi nhiều client cùng kết nối.

- Vòng đời: *afterConnectionEstablished* (validate token, lưu session, báo presence online) → *handleTextMessage* (parse JSON, kiểm quyền, xử lý theo type, lưu DB, broadcast) → *afterConnectionClosed* (xóa session, báo offline).
- **MessageType** (enum trong common-lib): CHAT, PRIVATE, SYSTEM, ERROR, JOIN, LEAVE, PING/PONG, EDIT, DELETE, TYPING, STATUS, REACT. Parse JSON bằng **Jackson** (có *JavaTimeModule* cho *LocalDateTime*).

E. Bất đồng bộ — RabbitMQ (Event-Driven)

- **Topic Exchange** *chat.exchange* (durable), định tuyến theo Routing Key. Bộ chuyển đổi *Jackson2JsonMessageConverter* để tự serialize JSON.
- Routing thực tế: *presence.status* (đổi trạng thái online/idle/dnd → messaging broadcast), *notification.** (tin mới → notification-service tính unread/@mention), *log.** (messaging đầy sự kiện → log-service ghi kiểm toán).
- Producer: *RabbitTemplate.convertAndSend(exchange, routingKey, msg)*. Consumer: *@RabbitListener(queues=...)* ở messaging, notification, log service.
- Lợi ích: tách tác vụ phụ (log/notify) khỏi luồng gửi tin chính → người dùng nhận phản hồi tức thì.

F. Dữ liệu — Spring Data JPA / Hibernate

- ORM **Hibernate** qua *spring-boot-starter-data-jpa*, MySQL 8 (dialect MySQL8), *ddl-auto=update* tự tạo schema.
- Entity chính: User, RefreshToken, ChatMessage, MessageReaction, Server, Member, Channel, Role, Notification, UnreadCounter, FileMetadata, UserProfile, Friendship.
- **Tối ưu**: đánh *@Index* trên channelId/sender/receiver (ChatMessage), userId+isRead (Notification); *@UniqueConstraint* (userId, channelId).
- **Soft-delete**: ChatMessage có cờ *isDeleted* — xóa tin chỉ set true, không xóa row.
- **Pagination**: trả *Page<T>* qua *PageRequest.of(page, size)*.

G. Lưu trữ tệp — MinIO

- SDK **io.minio 8.5.7**; bucket *chat-files*. Đặt tên tệp bằng **UUID** + đuôi mở rộng.
- Giới hạn **10MB**, kiểm tra MIME type (ảnh/video/pdf/doc/text). Metadata lưu bảng *file_metadata* (userId, channelId, fileName, size, mimeType, minioPath).
- **Thumbnail**: thư viện *Thumbnailator 0.4.20* tạo ảnh 200×200 lưu bucket *chat-thumbnails* → client tải nhanh hơn.

H. Presence — trạng thái hiện diện

- Lưu **in-memory** *ConcurrentHashMap<username, UserStatus>*. Trạng thái: ONLINE, IDLE, AWAY, DO_NOT_DISTURB, INVISIBLE, OFFLINE.
- Khi connect/disconnect/đổi status → publish sự kiện *presence.status* lên RabbitMQ → messaging broadcast cho mọi client cập nhật chấm trạng thái realtime.

I. DevOps — Docker, Jenkins, Giám sát

- **Dockerfile.template** tham số hóa (ARG SERVICE_NAME, EXPOSED_PORT): nền *eclipse-temurin:17-jre-alpine*, chạy **non-root** (appuser), JVM *UseContainerSupport, MaxRAMPercentage=75, UseG1GC*.
- **docker-compose.yml**: 12 service + MySQL/RabbitMQ/MinIO trên mạng *chat-net*, *depends_on* + *healthcheck* kiểm soát thứ tự khởi động; cấu hình qua biến môi trường (.env).
- **Jenkinsfile** (Declarative): Checkout → Build (mvn clean package) → build & push **song** **song** 12 image lên Docker Hub (tag = buildNumber-commitHash) → Deploy (docker compose up -d). Có sẵn stage SonarQube (đang tắt).
- **Giám sát**: Spring Boot Actuator expose */actuator/prometheus*; Prometheus scrape 10s; Grafana trực quan hóa. Micrometer cung cấp metrics JVM/HTTP.

J. Client — Java Swing + FlatLaf

- **Java Swing + FlatLaf 3.4** (theme Sáng/Tối, đổi nóng không cần khởi động lại). Đóng gói fat-JAR.
- Tầng network: *java.net.http.HttpClient* (Java 11+, singleton), *SessionManager* (lưu token, trường *volatile* + *synchronized*), các lớp **ApiClient* và *ChatWebSocketClient*.
- **Đa luồng UI**: tác vụ HTTP chạy nền bằng *SwingWorker*; cập nhật giao diện qua *SwingUtilities.invokeLater()* (đẩy về EDT) — tránh treo giao diện.
- **Cache**: *ImageCache* / *UserProfileCache* / *EmojiHelper* đều dùng *ConcurrentHashMap* để cache avatar, emoji, hồ sơ; tải ảnh bất đồng bộ, chống tải trùng.

K. Kỹ thuật Java cốt lõi (dễ bị hỏi)

- **OOP**: kế thừa (*TextWebSocketHandler*), đa hình (switch theo *MessageType*), đóng gói, trừu tượng (service interface).
- **Java 17 Record**: ví dụ *PagedResponse<T>* — lớp immutable ngắn gọn.
- **Đa luồng**: *ConcurrentHashMap* (7 nơi), từ khóa *volatile*, *synchronized* method/block.
- **Stream API** + **Optional** (*findBy...orElseThrow*), **Generics** (*JpaRepository<T, ID>*, *postJson<T>*).
- **Lombok**: *@Data*, *@Builder*, *@RequiredArgsConstructor* (KHÔNG dùng ở client-app). *@Transactional* đảm bảo toàn vẹn giao dịch.
- **Exception**: custom (*FileTooLargeException*, *ApiException*) + *@ExceptionHandler* trả mã lỗi HTTP. **Java Time**: *LocalDateTime* + *DateTimeFormatter*. **Regex**: *@(lw+)* để bắt *@mention*. **Log**: SLF4J + Logback.

L. Câu hỏi phản biện thường gặp (gợi ý trả lời)

Hỏi: Vì sao chọn Microservices mà không phải Monolith?

Đáp: Để mỗi nghiệp vụ (auth, messaging, file...) phát triển – triển khai – mở rộng độc lập; lỗi một service không sập toàn hệ thống; phù hợp mục tiêu môn học là thực hành kiến trúc phân

tán. Đánh đổi là phức tạp hơn về vận hành, nên nhóm bù lại bằng Docker Compose + giám sát Prometheus/Grafana.

Hỏi: Khác nhau giữa WebSocket và RabbitMQ trong dự án?

Đáp: WebSocket là kênh kết nối liên tục giữa *client* ↔ *server* để đẩy tin tức thời. RabbitMQ là message broker giữa các *service* (*server* ↔ *server*) theo kiểu bất đồng bộ, hướng sự kiện — dùng cho log, notification, đồng bộ trạng thái. Hai cái bổ trợ nhau, không thay thế.

Hỏi: JWT hết hạn thì xử lý thế nào? Có an toàn không?

Đáp: Access token sống 2 giờ; hết hạn thì client dùng refresh token (7 ngày, lưu DB) để xin token mới mà không phải đăng nhập lại. An toàn nhờ ký HMAC-SHA bằng secret phía server, mật khẩu hash BCrypt, và Gateway kiểm tra token tập trung trước khi vào service nội bộ.

Hỏi: Database-per-service thì làm sao truy vấn dữ liệu liên quan giữa các service?

Đáp: Không JOIN chéo DB. Khi cần dữ liệu của service khác thì gọi API qua OpenFeign (đồng bộ), hoặc lắng nghe sự kiện qua RabbitMQ (bất đồng bộ). Ví dụ messaging gọi server-service để kiểm tra quyền thành viên.

Hỏi: RBAC bitmask hoạt động ra sao, vì sao dùng bitmask?

Đáp: Mỗi quyền là một bit trong số nguyên; quyền của role là tổ hợp OR các bit. Kiểm tra quyền chỉ là phép AND bit — rất nhanh và gọn, lưu trữ chỉ 1 cột int thay vì nhiều dòng. Role gắn theo từng server.

Hỏi: Làm sao quản lý nhiều kết nối WebSocket đồng thời an toàn?

Đáp: Dùng ConcurrentHashMap lưu session theo username để thread-safe; mỗi tin nhắn được broadcast tới các session cùng kênh. Khi scale nhiều instance, có thể dùng RabbitMQ/Redis Pub-Sub để đồng bộ (đã nêu ở hướng phát triển).

Hỏi: Vì sao có common-lib? Có vi phạm tính độc lập của microservice không?

Đáp: common-lib chỉ chứa DTO/Enum/Util thuần (không chứa logic nghiệp vụ hay truy cập DB), nên chỉ là chuẩn hóa hợp đồng dữ liệu, giảm copy-paste (DRY). Các service vẫn độc lập về dữ liệu và triển khai.